

```

        public static void Test()
        {
            Application.Run(new AnotherSimpleForm2());
        }
    }

```

Surprisingly enough, these are all to the basics of Winforms. Forms contain controls. The look of a component can be changed via properties. The behaviour of a component can be changed via events.

7.3 The Winforms designer

Up to this point, we've created forms programatically, that is, at run-time. However, it is usually the case that most of our UI is static and events define its dynamic behavior. To this end, we'd like to design UIs as a step separate from the compilation process, ideally in a designer tailor-made for UI development. The winforms designer built into VS is one such tool.

How does it differ from what we've learned? Here is the designer code verbatim for the simple winforms app we wrote above.

```

#region Form1.cs (this is the code you can modify)
public partial class Form1 : Form
{
    int timesPressed;

    public Form1()
    {
        InitializeComponent();
    }

    private void button1_Click(object sender, EventArgs
e)
    {
        timesPressed += 1;
        label1.Text = "Hello: " + timesPressed;
    }
}
#endregion

```

```

#region Form1.Designer.cs (this is the code controlled
by the designer)

```